AgroSmart – Análisis de algoritmos y estructuras de datos

Cada algoritmo tiene: explicación, pseudocódigo, diagrama de flujo, complejidad en tiempo y espacio, justificación y un ejemplo real obtenido ejecutando el código del proyecto.

> Notación: **n** = número de elementos (cultivos, eventos, nodos). **V** = nodos del grafo, **E** = conexiones.

Dónde se usa cada cosa en la app

Elemento	Archivo	Pantalla donde se usa
Búsqueda lineal	`gestor_cultivos.py`	Cultivos → "Buscar cultivo"
Búsqueda binaria	`gestor_cultivos.py`	Cultivos → "Buscar por nombre exacto (rápido)"
Ordenamiento por inserción	`gestor_cultivos.py`	Cultivos → "Ordenar por fecha de siembra"
Merge Sort	`algoritmos/ordenamiento_clima.py`	Riego → "Repartir mi agua" (lo usa el algoritmo voraz) y `clima.py` (pronóstico ordenado por temperatura o lluvia, en su modo de prueba)
Dijkstra	`algoritmos/dijkstra.py`	Riego → "Camino más corto" y "Mejor orden para visitar mis pendientes"
Voraz (reparto de agua)	`algoritmos/voraz_agua.py`	Riego → "Repartir mi agua"
Tabla hash	`estructuras/tabla_hash.py`	Cultivos y Riego (buscar un cultivo por su código)
Pila	`estructuras/pila_movimientos.py`	Inventario → "Deshacer"
Cola	`estructuras/cola.py`	Riego → "Pendientes por regar"
Lista enlazada	`estructuras/lista_enlazada.py`	Riego → "Riegos ya hechos"
Árbol binario de búsqueda	`estructuras/arbol_eventos.py`	Calendario (eventos ordenados por fecha)
Grafo	`estructuras/grafo.py`	Riego → "Distancias y recorrido"

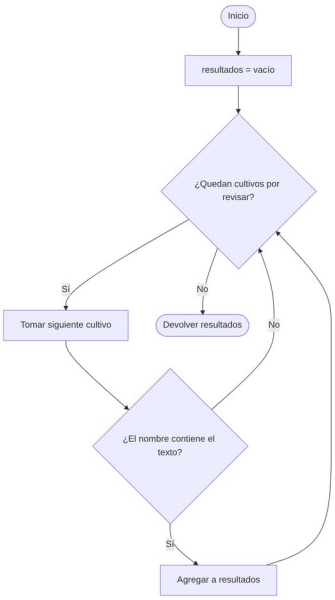
1. Búsqueda lineal (`GestorCultivos.buscar_por_nombre` / `buscar_coincidencias`)

Qué hace: revisa los cultivos uno por uno, desde el primero, y devuelve los que contienen el texto escrito. Sirve en el buscador de la pantalla Cultivos.

Pseudocódigo

```
funcion buscar_coincidencias(lista, texto):
    resultados = []
    para cada cultivo en lista:
        si texto está dentro de nombre(cultivo):
            agregar cultivo a resultados
    devolver resultados
```

Diagrama de flujo



Complejidad: tiempo O(n) (revisa todos). Espacio: O(1) adicional en `buscar_por_nombre` (devuelve un solo cultivo) y O(k) en `buscar_coincidencias`, donde k = cantidad de coincidencias (en el peor caso O(n)).

Justificación: la lista de cultivos de un agricultor es pequeña y no está ordenada por nombre; la búsqueda lineal no necesita preparar nada y permite buscar por *parte* del nombre ("fri" encuentra "frijol").

Ejemplo real: texto "frijol" → revisa 4 cultivos → encuentra "frijol parcela 9" y "frijol parcela 8".

---

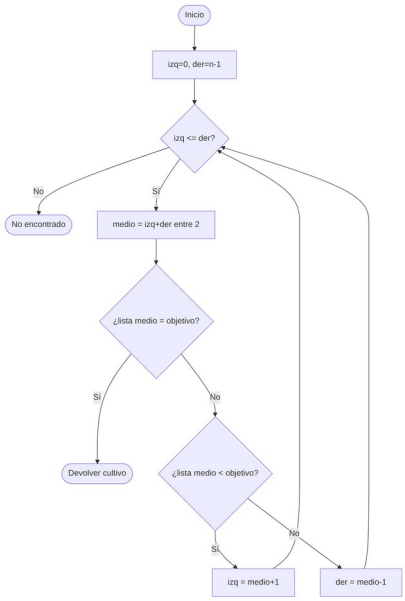
2. Búsqueda binaria (`GestorCultivos.buscar_por_nombre_binaria`)

Qué hace: con la lista **ordenada por nombre**, mira el elemento del medio; si no es, descarta la mitad donde no puede estar y repite.

Pseudocódigo

```
funcion buscar_binaria(lista_ordenada, objetivo):
    izq = 0 ; der = largo(lista_ordenada) - 1
    mientras izq <= der:
        medio = (izq + der) // 2
        si lista_ordenada[medio] == objetivo: devolver
lista_ordenada[medio]
        si lista_ordenada[medio] < objetivo:  izq = medio + 1
        sino:                                  der = medio - 1
    devolver NADA
```

Diagrama de flujo



Complejidad: tiempo O(log n); espacio O(n) por la copia ordenada que se crea (el algoritmo en sí usa O(1)).

Justificación: es mucho más rápida que la lineal cuando hay muchos cultivos y se busca el nombre exacto. Se usa una copia ordenada para no alterar el orden que ve el usuario.

Ejemplo real: lista ordenada `['frijol parcela 8','frijol parcela 9','maiz parcela 6','sandia parcela 6']`. "maiz parcela 6" → encontrado en **2 pasos**. "papa parcela 1" → no existe, termina en **3 pasos**.

---

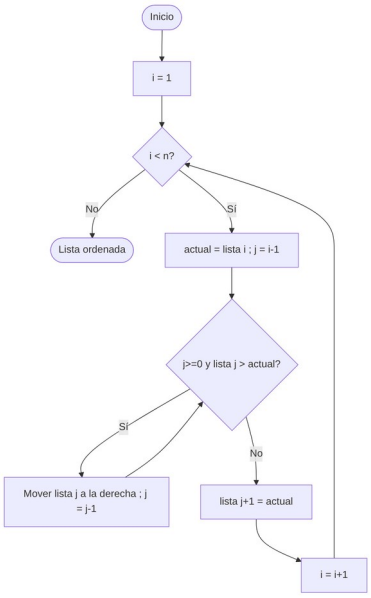
3. Ordenamiento por inserción (`GestorCultivos.ordenar_por_fecha_siembra`)

Qué hace: ordena los cultivos por fecha de siembra (formato AAAA-MM-DD, que se puede comparar como texto). Toma cada elemento y lo inserta en su lugar dentro de la parte ya ordenada.

Pseudocódigo

```
funcion ordenar_insercion(lista):
    para i desde 1 hasta largo(lista)-1:
        actual = lista[i]
        j = i - 1
        mientras j >= 0 y lista[j].fecha > actual.fecha:
            lista[j+1] = lista[j]
            j = j - 1
        lista[j+1] = actual
    devolver lista
```

Diagrama de flujo



Complejidad: tiempo $O(n^2)$ en el peor caso, $O(n)$ si ya está casi ordenada; espacio $O(1)$ (ordena dentro de la misma lista).

Justificación: pocos cultivos y, normalmente, nuevos cultivos con fechas cercanas al final: la inserción es simple y muy rápida en listas casi ordenadas, y es estable (respeta el orden de fechas iguales).

Ejemplo real: `["2026-09-08","2025-09-16","2025-09-06"]` → `["2025-09-06","2025-09-16","2026-09-08"]`.

---

4. Ordenamiento por mezcla – Merge Sort (`algoritmos/ordenamiento_clima.py`)

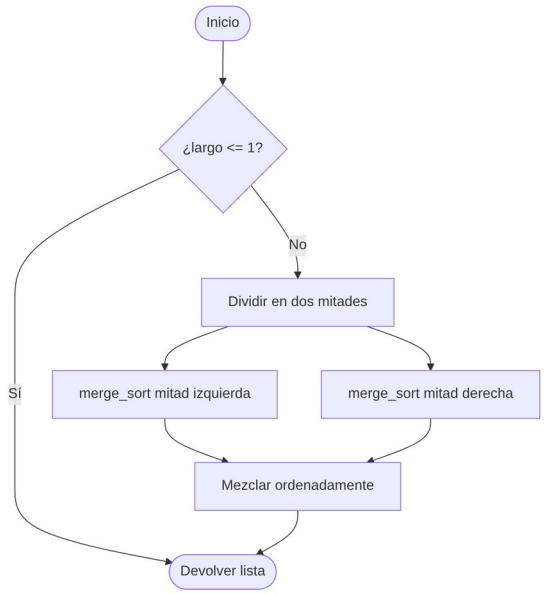
Qué hace: divide la lista a la mitad hasta tener elementos sueltos y luego los va mezclando de forma ordenada. Se usa para ordenar cultivos por beneficio al repartir agua (pantalla Riego) y para ordenar el pronóstico del clima por temperatura o por probabilidad de lluvia (módulo `clima.py`).

Pseudocódigo

```
funcion merge_sort(lista):
    si largo(lista) <= 1: devolver lista
    medio = largo(lista) // 2
    izquierda = merge_sort(lista[0..medio])
    derecha   = merge_sort(lista[medio..fin])
    devolver mezclar(izquierda, derecha)

funcion mezclar(a, b):
    resultado = []
    mientras a y b tengan elementos:
        mover el menor de los dos primeros a resultado
    agregar lo que sobre de a y de b
    devolver resultado
```

Diagrama de flujo



Complejidad: tiempo O(n log n) siempre (mejor, promedio y peor); espacio O(n) por las listas auxiliares.

Justificación: su rendimiento es predecible y no empeora con datos desordenados, ideal para ordenar los días del pronóstico que llegan del servicio web.

Ejemplo real: `[5,2,9,1]` → mezcla `[5]`[2]→`[2,5]`, `[9]`[1]→`[1,9]`, luego `[2,5]`+`[1,9]` → `[1,2,5,9]`.

---

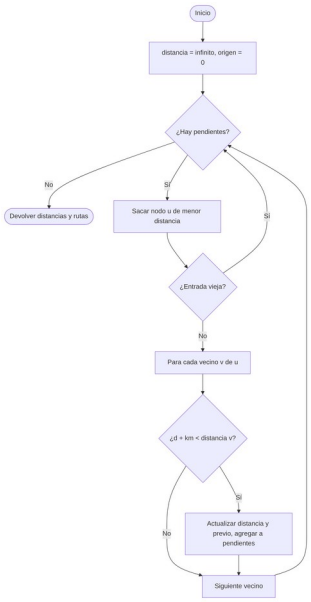
5. Algoritmo de grafos – Dijkstra (`algoritmos/dijkstra.py`)

Qué hace: encuentra el camino **más corto** (en kilómetros) entre dos lugares del grafo (casa y parcelas). Usa una cola de prioridad (heap).

Pseudocódigo

```
funcion dijkstra(grafo, origen):
    distancia[todos] = infinito ; distancia[origen] = 0
    previo[todos] = nada
    pendientes = [(0, origen)]
    mientras pendientes no esté vacío:
        (d, u) = sacar el de menor distancia
        si d > distancia[u]: continuar        # entrada vieja
        para cada (v, km) vecino de u:
            nueva = d + km
            si nueva < distancia[v]:
                distancia[v] = nueva ; previo[v] = u
                agregar (nueva, v) a pendientes
    devolver distancia, previo
```

Diagrama de flujo



Complejidad: tiempo O((V + E) log V); espacio O(V + E).

Justificación: las distancias entre parcelas son positivas, que es la condición de Dijkstra; con heap es eficiente y devuelve también el recorrido completo.

Ejemplo real: CASA–A 10 km, CASA–B 20 km, A–B 5 km → a B hay **15 km** por CASA → A → B (descarta la entrada vieja de 20 km).

---

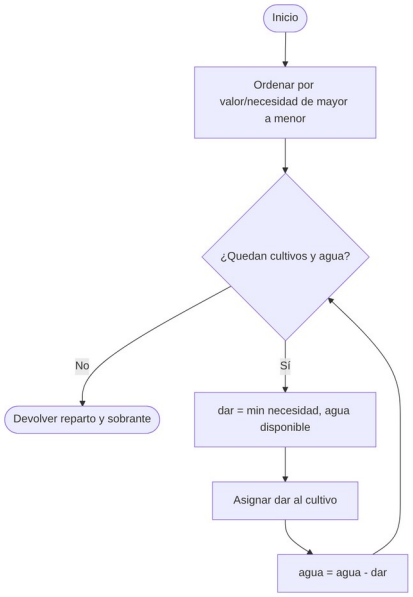
6. Algoritmo de optimización – Voraz (greedy) para repartir agua (`algoritmos/voraz_agua.py`)

Qué hace: con una cantidad limitada de agua, decide cuánto darle a cada cultivo. Ordena (con merge sort) por **beneficio por litro** (valor / necesidad) y reparte primero a los de mayor beneficio.

Pseudocódigo

```
funcion repartir_agua(cultivos, agua_disponible):
    ordenar cultivos de mayor a menor (valor / necesidad)   # merge_sort
    para cada cultivo en orden:
        dar = minimo(necesidad(cultivo), agua_disponible)
        asignar dar litros al cultivo
        agua_disponible = agua_disponible - dar
    devolver asignaciones y agua sobrante
```

Diagrama de flujo



Complejidad: tiempo O(n log n) (dominado por el ordenamiento); espacio O(n).

Justificación: es el problema de la mochila fraccionaria, donde la estrategia voraz **sí garantiza el óptimo**, y es muy rápida y fácil de explicar a un agricultor.

Ejemplo real (4000 L): frijol (0.01) 1500/1500 L, maíz (0.006) 2000/2000 L, sandía (0.003) 500/3000 L = 17 %, sobran 0 L.

Extras recomendados (breve)

Algoritmo	Dónde	Tiempo	Espacio
Búsqueda en tabla hash (código de cultivo)	`tabla_hash.py` / `GestorCultivos`	O(1) promedio, O(n) peor	O(n)
Recorrido in-order del árbol BST (calendario ordenado)	`arbol_eventos.py`	O(n)	O(h)
Inserción en BST	`arbol_eventos.py`	O(log n) promedio, O(n) peor	O(1)
Orden de visita (vecino más cercano)	`planificador.py`	$O(V^2)$	O(V)

Estructuras de datos: cómo funciona cada una

Pila (`PilaMovimientos`)

Idea: el último que entra es el primero que sale (LIFO), como una pila de platos.

```
apilar(x):    agregar x al final
desapilar():  si está vacía devolver NADA; si no, sacar y devolver el último
cima():       devolver el último sin sacarlo
```

Complejidad: apilar, desapilar y cima son O(1). Espacio O(n). **Uso:** el botón "Deshacer" del Inventario revierte siempre el movimiento más reciente.

Cola (`Cola`)

Idea: el primero que entra es el primero que sale (FIFO), como la fila de una tienda. Está hecha con nodos enlazados: guarda un puntero al primero y otro al último.

```
encolar(x):    crear nodo; si la cola está vacía, primero = último = nodo;
               si no, último.siguiente = nodo y último = nodo
desencolar():  si está vacía devolver NADA; si no, sacar el primero y avanzar el puntero
```

Complejidad: encolar y desencolar O(1). Espacio O(n). **Uso:** "Pendientes por regar": se riega en el orden en que se agregaron.

Lista enlazada (`ListaEnlazada`)

Idea: una cadena de nodos; cada nodo guarda un dato y apunta al siguiente.

```
agregar_al_inicio(x):  nodo.siguiente = cabeza; cabeza = nodo
agregar_al_final(x):   recorrer hasta el último nodo y enlazar el
nuevo
a_lista():             recorrer desde la cabeza copiando cada dato
```

Complejidad: agregar al inicio O(1); agregar al final y recorrer O(n). Espacio O(n). **Uso:** "Riegos ya hechos": cada riego nuevo se agrega al inicio, así el más reciente sale primero.

Árbol binario de búsqueda (`ArbolEventos`)

Idea: cada nodo tiene un hijo izquierdo (fechas menores) y uno derecho (fechas mayores o iguales).

```
insertar(evento):  desde la raíz, ir a la izquierda si la fecha es
menor, si no a la derecha,
                   hasta encontrar un lugar vacío y colocarlo ahí
en_orden():        recorrer izquierda, luego el nodo, luego derecha
```

Complejidad: insertar O(log n) en promedio y O(n) si el árbol queda inclinado; recorrido en orden O(n). Espacio O(n). **Uso:** el Calendario muestra los eventos ordenados por fecha sin tener que reordenar la lista.

Grafo (`Grafo`)

Idea: lugares (nodos) conectados por caminos con una distancia en km (aristas). Se guarda como un diccionario de listas de adyacencia: para cada lugar, la lista de sus vecinos y la distancia a cada uno.

```
agregar_nodo(a):       si a no existe, crear su lista de vecinos vacía
agregar_arista(a,b,km): guardar b en los vecinos de a y a en los de b,
con km
existe(a):             devolver si a está en el grafo
```

Complejidad: agregar nodo y arista O(1); espacio O(V + E). **Uso:** "Distancias y recorrido": la casa y las parcelas son los nodos; Dijkstra calcula el camino más corto.

Resumen de las estructuras

Estructura	Archivo	Para qué se usa en la app	Insertar / buscar
Arreglo / lista	`gestor_cultivos.py`	Guarda todos los cultivos	O(1) al final / O(n)
Tabla hash	`tabla_hash.py`	Encontrar un cultivo por su código al instante	O(1) prom.
Pila	`pila_movimientos.py`	Botón "Deshacer" en Inventario (último movimiento primero)	O(1)
Cola	`cola.py`	Riegos pendientes (el	O(1)

Estructura	Archivo	Para qué se usa en la app	Insertar / buscar
		primero en llegar se atiende primero)	
Lista enlazada	`lista_enlazada.py`	Historial de riegos ya hechos	O(1) al inicio / O(n)
Árbol binario de búsqueda	`arbol_eventos.py`	Eventos del calendario ordenados por fecha	O(log n) prom.
Grafo	`grafo.py`	Lugares y distancias en km entre casa y parcelas	O(1) agregar arista